perplexity

AI gateway security architecture: outline an enterprise specification with boundary enforcements, DLP filters, and student-record redaction rules

Computer
AI gateway policy enforcement matrix: build an implementation blueprint for request validation, token guardrails, and data boundary controls
Computer
Show me the code for boundary 2 data minimization
Continue with the remaining boundaries in the gateway
Map out the middleware pattern for the execution pipeline

Below is an enterprise implementation blueprint for Semester's AI gateway, including boundary enforcement, student-record exclusion rules, data-loss prevention (DLP), TypeScript code for data minimization, and the execution middleware pattern.

The core requirement is **exclusion**: protected student records must not reach a model provider—even when the requesting user is authorized to view them inside Semester.

This is a proposed implementation specification, not a claim that these controls are already deployed or verified.

1. Establish one canonical boundary map

Your latest request calls data minimization "boundary 2." The previous outline called authorization boundary 2. To avoid inconsistent code, documentation, and audit events, use the following numbering going forward:

Boundary	Responsibility	Enforcement point
B1	Request validation, identity, environment, and authorization	API ingress and policy service
B2	Data minimization and student-record exclusion	Authorized data-access layer and DTO builder
B3	Retrieval, provenance, and freshness	Retrieval broker
B4	Provider egress, DLP, and token budgets	Provider adapter
B5	Tool proposals and controlled actions	Command gateway
B6	Output validation and authorized delivery	Response gateway

Authorization is not performed only in B1: repeat object-level checks at retrieval, commands, and delivery.

Next.js recommends centralizing authorization in a data-access layer and using data-transfer objects to return only necessary data; route middleware is not a substitute for checks close to data access. [1] [2]

Execution flow

```
Untrusted browser request
    |
    ▼
B1: Validate request → verify session → resolve scope → authorize
    |
    ▼
B2: Exclude protected sources → select safe fields → build minimal DTO
    |
    ▼
B3: Retrieve permitted evidence → check freshness → attach provenance
    |
    ▼
B4: Scan complete provider payload → enforce budgets → approved provider
    |
    ▼
Model output — treated as untrusted
    |
    ├── tool proposal → B5 → approval/command gateway → safe tool result
    |                                                      |
    |                          rerun B2–B4 before another provider call
    ▼
B6: Validate output → recheck delivery authority → release response
```

A model response is never an authorization decision, approved command, SQL statement, or trusted HTML. OWASP recommends treating model output as untrusted input and validating it before downstream use. [3]

2. Enterprise policy-enforcement matrix

Control	Required behavior	Failure behavior	Audit evidence
Request schema	Strict JSON schema; reject unknown fields and unsupported modes	Reject before retrieval	Schema version, reason code
Request size	Enforce byte limits before JSON parsing	413	Byte count, no body
Session	Verify authenticity, expiry, revocation, and intended audience	401	Session reference
Workspace	Derive authority server-side; validate selected workspace against grants	403	Scope and decision ID
Environment	Bind deployment, credentials, data stores, queues, and provider identity	Block invocation	Environment configuration version
Source authorization	Apply tenant/object/field rules before retrieval	Exclude or deny	Retrieval decision ID

Control	Required behavior	Failure behavior	Audit evidence
Student-record exclusion	Reject protected sources regardless of actor permissions	No provider call	Exclusion category
Free-text DLP	Inspect prompt, evidence, attachments, history, tool results, and output	Block ambiguous sensitive content	Detector version and category
Token budget	Count complete compiled payload with model-compatible tokenizer	Reject or rebuild approved context	Input/output budget
Provider routing	Only approved model/provider configurations	No arbitrary fallback	Provider policy version
Tools	Validate proposal; apply full command authorization and approval	No side effect	Command and approval references
Delivery	Revalidate permissions and session after generation	Withhold response	Delivery decision
Logging	Metadata only by default; no raw sensitive text	Disable body capture	Audit schema version

For all boundaries:

- Unknown classification means deny.
- Missing policy means deny.
- Unavailable required DLP or policy service means block, not bypass.
- Provider fallback must satisfy the same controls.
- Errors must not reveal whether an inaccessible student record exists.

3. Student-record redaction and exclusion rules

Rule A: authorization does not permit provider disclosure

A student may view their own grade in Semester. That does not authorize sending that grade to a model provider.

Keep two separate decisions:

```
May this actor view the record in Semester?
May this information leave Semester for model processing?
```

The second answer remains “no” for protected student records under this architecture.

Rule B: exclude whole protected records

Do not transform this:

```
{
  "student_name": "Example Student",
  "student_id": "S123",
  "grade": "C",
```

```
  "hold": "financial",
  "accommodation": "extended time"
}
```

into this and call it safe:

```
{
  "student_name": "[REDACTED]",
  "grade": "C",
  "hold": "financial",
  "accommodation": "extended time"
}
```

The remaining facts still originate from a protected record. Removing direct identifiers does not meet Semester's stronger exclusion policy.

Protected-source content includes:

- Names, student identifiers, institutional emails, and contact details.
- Grades, transcripts, enrollment, attendance, and advising records.
- Holds, account balances, aid decisions, and payment details linked to students.
- Accommodation, counseling, health, conduct, or disciplinary information.
- Student-linked derived facts, including "eligible," "at risk," or "hold cleared."
- Embeddings, excerpts, summaries, and model-generated descriptions derived from those records.

These are architectural exclusion categories, not a legal determination that every listed item has the same statutory classification.

Rule C: generic assistance and personal status use separate channels

For "Can I register now?":

1. A deterministic Semester service evaluates eligibility.
2. The authorized UI displays that result.
3. The model may explain generic registration policy from permitted documentation.
4. The personal eligibility result is not sent to the provider.

```
Protected record → authorized deterministic service → private UI card
Public policy    → AI gateway → generic explanation
```

Do not feed the private UI card back into conversational history.

Rule D: unstructured content is not automatically safe

Student information can appear in:

- User-entered questions.
- Support tickets.
- Uploaded documents.
- Image OCR.
- Tool responses.
- URLs and query strings.
- Retrieved internal notes.
- Previous conversation turns.

DLP must inspect all provider-bound surfaces. Regexes can detect some obvious patterns, but cannot establish that arbitrary prose contains no student-record facts. Unknown or ambiguous content requires blocking, a narrower supported interaction, or human review.

4. Boundary 2: data-minimization code

The following TypeScript shows the structural enforcement. It intentionally excludes protected records rather than trying to anonymize them.

It is a reference module, not a complete production DLP implementation.

Types and policy contract

```
// gateway/minimize.ts
import "server-only";
import { z } from "zod";

export class GatewayError extends Error {
  constructor(public readonly code: string) {
    super(code);
  }
}

type Classification =
  | "public"
  | "internal"
  | "protected_student_record"
  | "payment_sensitive"
  | "secret"
  | "unknown";

type Destination = "model_provider";

export type ExecutionContext = Readonly<{
  actorId: string;
  tenantId: string;
  workspaceId: string;
```

```
  environment: "local" | "preview" | "staging" | "production";
  sessionId: string;
}>;

export type EgressGrant = Readonly<{
  decisionId: string;
  expiresAt: number;
  actorId: string;
  tenantId: string;
  workspaceId: string;
  environment: ExecutionContext["environment"];
  destination: Destination;
  allowedClasses: readonly ("public" | "internal")[];
  maxDocuments: number;
  maxTextBytes: number;
}>;

// Loaded by the server-side repository.
// These labels must not come from the browser or model.
export type ClassifiedDocument = Readonly<{
  tenantId: string;
  workspaceId: string;
  classification: Classification;
  lineageClasses: readonly Classification[];
  publication: "approved" | "draft";
  revision: string;
  providerExcerpt: string;
}>;

export type DlpScanner = {
  inspect(
    text: string,
    options: { destination: Destination }
  ): Promise<{ safe: boolean; categories: string[] }>;
};

const SafeEvidenceSchema = z.object({
  evidenceId: z.string(),
  revision: z.string(),
  text: z.string(),
}).strict();

export type SafeEvidence = z.infer<typeof SafeEvidenceSchema>;

const FORBIDDEN = new Set<Classification>([
  "protected_student_record",
  "payment_sensitive",
  "secret",
  "unknown",
]);

function assertGrant(
  ctx: ExecutionContext,
  grant: EgressGrant
): void {
  if (
```

```
    grant.expiresAt <= Date.now() ||
    grant.actorId !== ctx.actorId ||
    grant.tenantId !== ctx.tenantId ||
    grant.workspaceId !== ctx.workspaceId ||
    grant.environment !== ctx.environment ||
    grant.destination !== "model_provider"
  ) {
    throw new GatewayError("INVALID_EGRESS_GRANT");
  }
}
```

Minimal DTO builder

```
export async function minimizeForProvider(
  ctx: ExecutionContext,
  grant: EgressGrant,
  documents: readonly ClassifiedDocument[],
  dlp: DlpScanner
): Promise<SafeEvidence[]> {
  assertGrant(ctx, grant);

  if (documents.length > grant.maxDocuments) {
    throw new GatewayError("CONTEXT_DOCUMENT_LIMIT");
  }

  const safe: SafeEvidence[] = [];
  let usedBytes = 0;

  for (const doc of documents) {
    if (
      doc.tenantId !== ctx.tenantId ||
      doc.workspaceId !== ctx.workspaceId
    ) {
      throw new GatewayError("SCOPE_MISMATCH");
    }

    if (
      FORBIDDEN.has(doc.classification) ||
      doc.lineageClasses.some(c => FORBIDDEN.has(c))
    ) {
      throw new GatewayError("PROHIBITED_SOURCE");
    }

    if (
      !grant.allowedClasses.includes(
        doc.classification as "public" | "internal"
      ) ||
      doc.publication !== "approved"
    ) {
      throw new GatewayError("SOURCE_NOT_APPROVED");
    }

    const text = doc.providerExcerpt.normalize("NFC");

    // Fail closed if the scanner throws or returns unsafe.
```

```
    const scan = await dlp.inspect(text, {
      destination: "model_provider",
    });

    if (!scan.safe) {
      throw new GatewayError("DLP_BLOCKED");
    }

    usedBytes += Buffer.byteLength(text, "utf8");

    if (usedBytes > grant.maxTextBytes) {
      throw new GatewayError("CONTEXT_BYTE_LIMIT");
    }

    // Construct from an allowlist; never spread the source object.
    safe.push(SafeEvidenceSchema.parse({
      evidenceId: `E${safe.length + 1}`,
      revision: doc.revision,
      text,
    }));
  }

  return safe;
}
```

Important implementation requirements

- Obtain `EgressGrant` from the policy service, never the client.
- Select only safe source columns in the repository query.
- Use RLS and server-side source authorization before this function runs.
- Maintain trusted lineage classifications: a summary derived from a student record is still forbidden.
- Keep real source IDs in a private evidence map; send request-local handles such as `E1` to the provider.
- Do not log `documents`, `text`, or DLP matches.
- Do not silently truncate content to fit budgets.

The code checks scope, but scope checks alone do not replace object-level authorization. The repository must enforce that before returning documents.

5. Boundary 1: request validation and trusted context

The browser submits intent, not identity or authority.

```
import { z } from "zod";

export const AskSchema = z.object({
  question: z.string().trim().min(1).max(4000),
  workspace: z.string().min(1).max(80),
```

```
  conversationId: z.string().uuid().optional(),
}).strict();
```

Ingress must also enforce:

- Bounded body reading before `JSON.parse`.
- Supported content type.
- Authentication and active session checks.
- CSRF/origin protections appropriate to cookie-authenticated requests.
- Tenant-scoped conversation ownership.
- Per-actor and tenant rate limits.
- Server-resolved workspace grants.
- Request-purpose allowlisting.

Reject client fields such as `actorId`, `tenantId`, `permissions`, `providerUrl`, `systemPrompt`, or `serviceRole`.

A client-selected workspace is acceptable only as a selection hint that the server validates against active memberships.

6. Boundary 3: authorized retrieval and freshness

The retrieval broker must:

1. Search only permitted source classes.
2. Apply access rules before search and ranking.
3. Return bounded excerpts.
4. Revalidate source classifications and lineage.
5. Attach source revision, retrieval decision, and freshness.
6. Keep sensitive source references out of provider payloads.

```
type EvidenceMetadata = {
  evidenceId: string;
  sourceRef: string;          // Server-private.
  revision: string;
  retrievalDecisionId: string;
  retrievedAt: string;
  freshness: "fresh" | "stale" | "unknown";
};
```

Do not search a global mixed-permission vector index and then rely only on post-retrieval redaction. A provider-bound index must exclude protected student-record embeddings as well as protected text.

Freshness rules should be capability-specific. A stale published help article might be usable with a warning; a stale eligibility projection must not be used to assert a student's current

status.

7. Boundary 4: DLP, token guardrails, and provider egress

Scan the complete compiled request

Field-level scans are insufficient. B4 must inspect the actual final payload, including:

- System instructions.
- User question.
- Approved conversation history.
- Retrieved evidence.
- Tool descriptions.
- Tool results.
- Attachment-derived text.
- Any metadata transmitted to the provider.

Context assembly can expose combinations of information that individual field scans miss.

Token controls

Use the configured model's tokenizer—not character estimates—for enforcement.

```
type TokenBudget = {
  maxInputTokens: number;
  maxOutputTokens: number;
  safetyMarginTokens: number;
  modelContextTokens: number;
};

function enforceTokenBudget(
  inputTokens: number,
  budget: TokenBudget
): void {
  if (
    inputTokens > budget.maxInputTokens ||
    inputTokens +
      budget.maxOutputTokens +
      budget.safetyMarginTokens >
      budget.modelContextTokens
  ) {
    throw new GatewayError("TOKEN_BUDGET_EXCEEDED");
  }
}
```

Also enforce:

- Maximum model invocations per request.

- Maximum tool rounds.
- Per-tenant concurrency.
- Request deadline and provider timeout.
- Daily spend limits.
- Explicit cancellation propagation.

Never remove system safety instructions to make a request fit. Rebuild evidence under a smaller approved budget or reject the request.

Provider adapter controls

The adapter owns:

- Fixed, approved provider endpoints.
- Approved model IDs.
- Environment-bound credentials.
- Retention and training settings verified against applicable agreements.
- No automatic downgrade to an unapproved provider.
- No payload-body logging.
- Final egress scan.
- A single network invocation path.

A TypeScript “safe payload” type helps prevent mistakes, but is not a security boundary by itself. Enforce network restrictions so unrelated code cannot bypass the gateway.

8. Boundary 5: tool mediation and approvals

The model can emit a proposal:

```
const ProposalSchema = z.object({
  tool: z.literal("create_support_case"),
  input: z.object({
    subject: z.string().max(120),
    summary: z.string().max(1000),
  }).strict(),
}).strict();
```

The command gateway then:

1. Validates the proposal.
2. Resolves target objects server-side.
3. Rechecks session and current permissions.
4. Applies the 14-check command authorization chain.
5. Requires approval where policy demands it.

6. Binds approval to the exact command arguments and target.
7. Executes idempotently.
8. Writes domain state, audit, and outbox atomically.
9. Returns a minimal result.

A tool result containing protected data stays inside Semester. It does not become model context.

If the proposal requires protected identifiers, resolve them from an authorized server-side context or a user-approved UI selection—not from guessed model identifiers.

9. Boundary 6: output validation and safe delivery

Before returning generated text:

- Validate its response schema.
- Scan for sensitive information and secret leakage.
- Confirm referenced evidence handles exist.
- Reject unsupported tool-execution claims.
- Recheck session and delivery permissions.
- Render as inert text or sanitized Markdown.
- Restrict link schemes and external links.
- Record completion metadata without recording sensitive body text.

OWASP identifies insufficient validation of LLM outputs as a downstream security risk and recommends context-aware output handling, safe rendering, and parameterized database operations.[3]

For the initial sensitive gateway implementation, buffer the response until validation finishes. Sending raw provider tokens to the browser and checking them afterward cannot prevent disclosure.

Citation validation establishes that references exist; it does not prove every generated statement is supported. High-impact answers need stronger evidence checks or deterministic templates.

10. Middleware pattern for the execution pipeline

Use two different concepts:

Layer	Purpose
Next.js request middleware/proxy	Coarse routing, basic request controls, navigation checks
Gateway execution middleware	Ordered, server-side enforcement around retrieval, provider calls, tools, and delivery

Do not make framework routing middleware the sole authority boundary. The critical checks belong in server handlers and data-access services.[2] [1]

Ordered pipeline interface

```
type Stage = (state: PipelineState) => Promise<void>;

type PipelineState = {
  requestId: string;
  rawRequest: Request;
  input?: z.infer<typeof AskSchema>;
  context?: ExecutionContext;
  grant?: EgressGrant;
  evidence?: SafeEvidence[];
  providerPayload?: unknown;
  candidate?: unknown;
  response?: {
    text: string;
    evidenceIds: string[];
  };
};

type GatewayServices = {
  audit: {
    stageStarted(requestId: string, stage: string): Promise<void>;
    stageCompleted(requestId: string, stage: string): Promise<void>;
    requestFailed(requestId: string, code: string): Promise<void>;
  };
};

export async function runPipeline(
  state: PipelineState,
  stages: readonly { name: string; run: Stage }[],
  services: GatewayServices
) {
  try {
    for (const stage of stages) {
      await services.audit.stageStarted(state.requestId, stage.name);
      await stage.run(state);
      await services.audit.stageCompleted(state.requestId, stage.name);
    }

    if (!state.response) {
      throw new GatewayError("MISSING_VALIDATED_RESPONSE");
    }

    return state.response;
  } catch (error) {
    const code = error instanceof GatewayError
      ? error.code
      : "GATEWAY_UNAVAILABLE";

    await services.audit.requestFailed(state.requestId, code);
    throw new GatewayError(code);
  }
}
```

Assembly

```
const stages = [
  { name: "B1", run: validateAndAuthorize },
  { name: "B2", run: retrieveAndMinimizeSources },
  { name: "B3", run: validateEvidenceAndFreshness },
  { name: "B4", run: compileScanAndInvokeProvider },
  { name: "B5", run: mediateToolProposals },
  { name: "B6", run: validateAndAuthorizeDelivery },
] as const;
```

The stage names describe boundaries, not permission to skip checks. `mediateToolProposals` must call the same minimization and provider-egress services before every additional model invocation.

In production, prefer typed stage outputs over a broadly mutable shared state. The simplified interface above illustrates execution order; stage contracts must reject missing prerequisites.

Route handler

```
export async function POST(request: Request) {
  const requestId = crypto.randomUUID();

  try {
    const result = await runPipeline(
      { requestId, rawRequest: request },
      stages,
      services
    );

    return Response.json(
      { requestId, ...result },
      { headers: { "Cache-Control": "no-store" } }
    );
  } catch {
    return Response.json(
      {
        requestId,
        state: "blocked",
        message: "This request could not be completed safely.",
      },
      {
        status: 503,
        headers: { "Cache-Control": "no-store" },
      }
    );
  }
}
```

The example uses one safe fallback. Production error mapping should distinguish malformed requests, unauthenticated requests, denials, budget limits, and unavailable dependencies—without exposing restricted details.

11. Required verification before enabling production

Test	Expected outcome
Authorized student asks to send their own transcript to the provider	Blocked
Public-looking summary has protected-record lineage	Blocked
Prompt includes a student ID or sensitive student narrative	Blocked or safely redirected
Unknown source classification	Blocked
DLP service times out	No provider call
Tool result contains a student grade	Private result only; excluded from next model call
Client spoofs actor or tenant fields	Strict-schema rejection
Session revoked during generation	Response withheld
Model returns executable HTML or script	Never executed
Model invents evidence ID	Response rejected
Preview uses production credentials	Startup/invocation blocked
Complete compiled prompt exceeds token budget	No provider call
Provider fallback violates approved routing	Blocked
Raw output streaming attempts to bypass B6	Blocked

The acceptance criterion is not merely “DLP found common identifiers.” It is that all provider traffic passes through one fail-closed egress path, protected sources are structurally excluded, and every answer and action remains subject to Semester’s current authorization rules.

⁂

1. https://nextjs.org/docs/app/guides/authentication
2. https://nextjs.org/docs/14/app/building-your-application/authentication
3. https://genai.owasp.org/llmrisk/llm05-supply-chain-vulnerabilities/
4. projects.semester.ai_gateway_governance
5. https://www.perplexity.ai/search/a528cbc7-358a-4556-b818-4c908c1e9f50
6. https://www.perplexity.ai/search/bf2642de-a9ce-4877-be30-7275fdbf851d
7. https://supabase.com/docs/guides/database/postgres/row-level-security
8. https://cheatsheetseries.owasp.org/cheatsheets/LLM_Prompt_Injection_Prevention_Cheat_Sheet.html
9. https://supabase.com/features/row-level-security
10. https://genai.owasp.org/llmrisk/llm01-prompt-injection/
11. https://makerkit.dev/blog/tutorials/supabase-rls-best-practices
12. https://www.brightdefense.com/resources/owasp-top-10-llm/
13. https://www.vectra.ai/topics/prompt-injection
14. https://egghead.io/lessons/supabase-use-the-supabase-service-key-to-bypass-row-level-security

15. https://www.robinwieruch.de/next-authorization/

16. https://focusreactive.com/blog/nextjs-protected-routes-middleware-jwt/

17. https://timomeh.de/posts/if-youre-using-next-js-middleware-for-authorization-youre-doing-something-wrong

18. https://github.com/vercel/next.js/discussions/63775